

CAMPUS COIN

Smart Spending Student Style

End-to-End Web Solutions
Project Documentation

Based on Software Requirements Specification (SRS) Version 1.0

Prepared for Project Evaluation / Viva

Table of Contents

1. Problem Definition
2. Design Specifications – ER, DFDs and Flowcharts
3. System Architecture
4. Security Features
5. Wireframes / Mockups
6. Database Design
7. Test Data
8. Project Installation Instructions
9. Functional Test Cases
10. Assumptions and Scope Notes
11. AI Usage Acknowledgement
12. Conclusion

1. Problem Definition

Campus Coin is designed to address the difficulty college and university students face when tracking money received from irregular sources and monitoring everyday spending. The SRS identifies allowances, part-time/gig income, scholarships and gifts as common income sources, while food, transport, hostel/rent, academics, subscriptions and entertainment are common student expenses.

The proposed solution is a lightweight, responsive student-first web application that allows users to record income and expenses, categorize transactions, view spending patterns, set budgets, receive saving tips, and optionally use AI assistance for categorization and monthly insights.

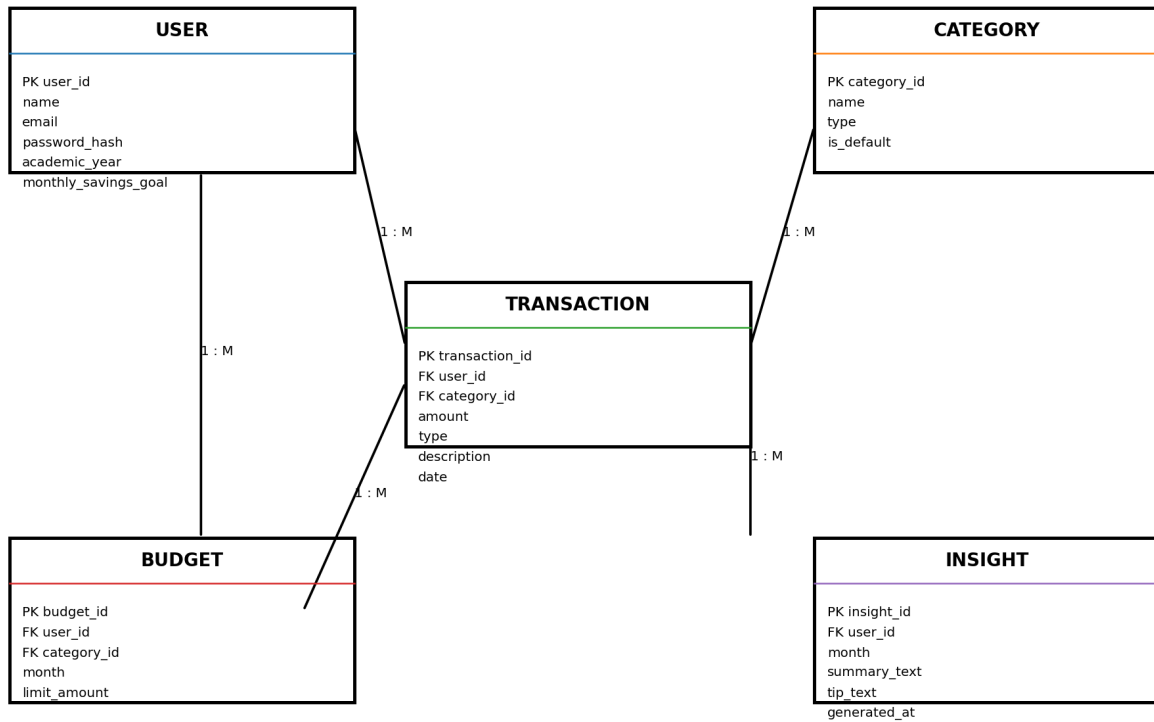
- Fast entry of income and expenses using student-relevant categories.
- Personalized saving tips based on transaction history.
- Optional AI-assisted expense categorization and monthly spending summaries.
- Dashboard charts and reports for understanding spending behavior.
- Budget goals and alerts for category-level spending.
- Administrative oversight of users, categories and system usage.

2. Design Specifications

2.1 Entity Relationship Design

Entity	Key Fields	Relationship
User	user_id, name, email, password_hash, academic_year, monthly_savings_goal	One user can own many transactions, budgets and insights.
Category	category_id, name, type, is_default	A category can be used by many transactions and budgets.
Transaction	transaction_id, user_id, category_id, amount, type, description, date	Belongs to one user and one category.
Budget	budget_id, user_id, category_id, month, limit_amount	Belongs to one user and one category for a month.
Insight	insight_id, user_id, month, summary_text, tip_text, generated_at	Belongs to one user and represents a monthly insight.

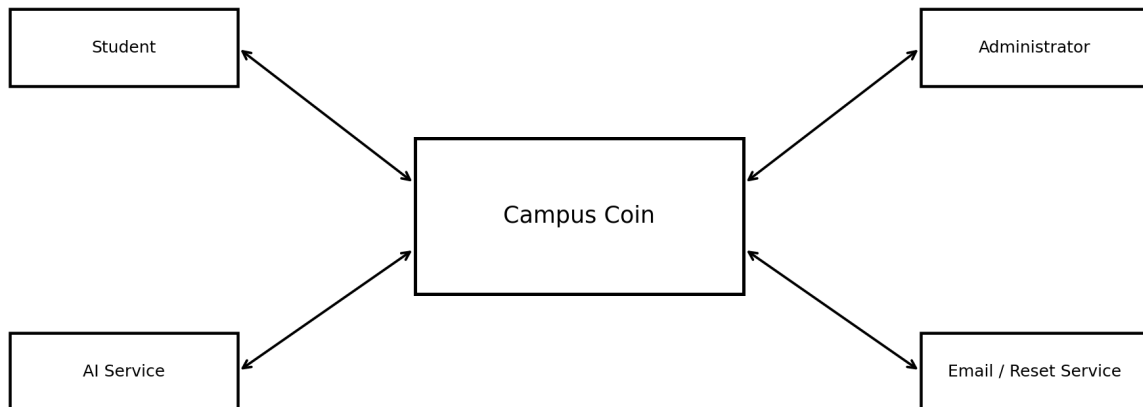
Entity Relationship Diagram – Campus Coin



2.2 Data Flow Diagrams

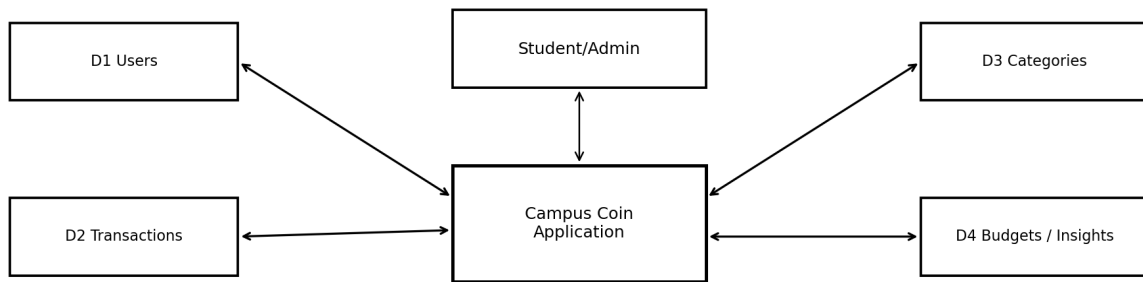
DFD Level 0 / Context Diagram

DFD Level 0 / Context Diagram



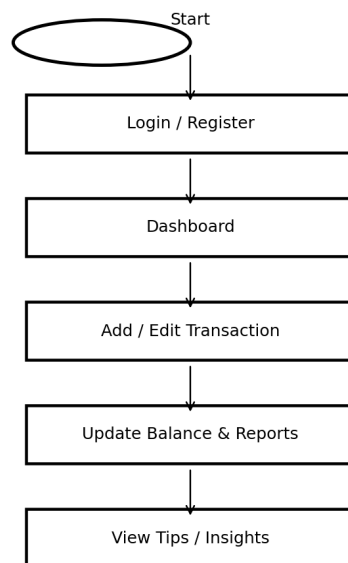
DFD Level 1 – Major Data Flows

DFD Level 1 – Major Data Flows



2.3 Flowchart

Student Transaction & Insight Flowchart



Typical flow: Start → Login/Register → Dashboard → Add/Edit Transaction → Update Balance & Reports → View Saving Tips/Insights.

3. System Architecture

The SRS describes a standard three-tier architecture: Presentation Layer, Application/API Layer and Data Layer. The Presentation Layer contains the responsive web front end, the Application/API Layer handles authentication, transaction logic, reporting and optional AI requests, and the Data Layer stores users, transactions, categories, budgets and generated insights.

3.1 Frontend, Backend, Database and APIs

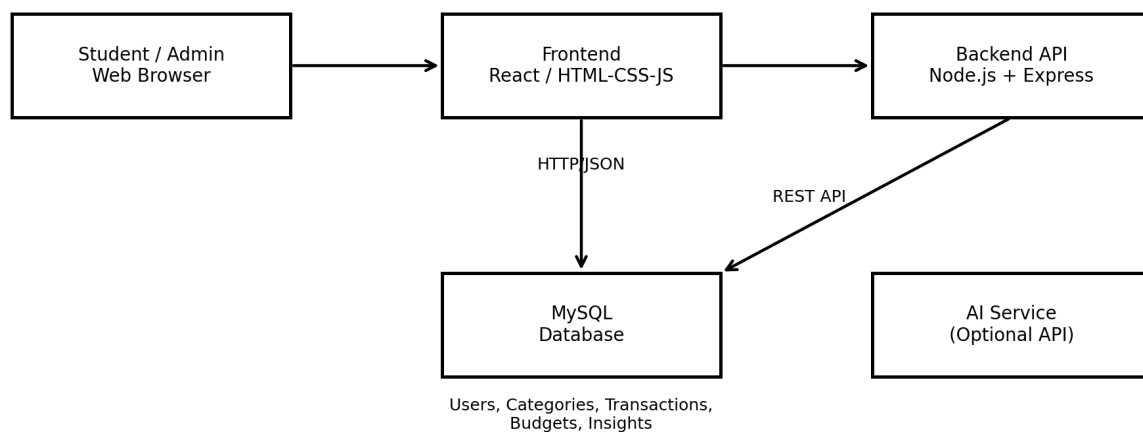
The SRS permits several technology combinations and does not mandate one exact stack. For this documentation, React + Node.js/Express + MySQL is used as an implementation assumption; replace it if the actual submitted project

uses another SRS-approved stack.

Layer	Documented Stack / Component	Purpose
Frontend	React + HTML5 + CSS3 + JavaScript	Responsive dashboard, forms, charts and navigation.
Backend	Node.js + Express.js	REST API, authentication, validation and business logic.
Database	MySQL	Persistent storage.
AI Integration	Optional external AI API	Expense categorization and monthly insights.
Authentication	Session/token based	Protect accounts and user data.
Reports	Backend report service	Monthly and category summaries.

3.2 Architecture Diagram

Campus Coin – Overall System Architecture



3.3 API Interaction

API Area	Example Endpoint	Purpose
Authentication	POST /api/auth/register	Create student account.
Authentication	POST /api/auth/login	Authenticate student/admin.
Transactions	GET/POST /api/transactions	Read/create transactions.
Transactions	PUT/DELETE /api/transactions/:id	Edit/delete transaction.
Categories	GET/POST /api/categories	Manage categories.
Budgets	GET/POST /api/budgets	Manage monthly budgets.
Reports	GET /api/reports/monthly	Generate monthly report.
Insights	GET/POST /api/insights	Read/generate insights.
Admin	GET /api/admin/stats	View usage statistics.

Endpoint names are documentation examples because the SRS specifies capabilities rather than exact endpoint URLs.

4. Security Features

- Authentication for registered users.
- Password hashing instead of plain-text password storage.
- Role-based authorization for student/admin functions.
- Session/token expiration and protected routes.
- Server-side input validation.
- Parameterized queries/prepared statements against SQL injection.
- Safe output encoding against XSS.
- CSRF protection when cookie-based authentication is used.
- Time-limited password reset tokens.
- CSV upload validation for type, size, headers and values.
- Isolation of each student's transaction history.
- HTTPS/TLS for production deployment.
- Restricted administrative controls.

5. Wireframes / Mockups

Screen	Main UI Components	Purpose
Home / Landing	Logo, summary, Login/Register, features, sitemap	Introduction and navigation.
Login	Email, password, forgot password	Secure access.
Registration	Name, email, password, academic year, savings goal	Create account.
Dashboard	Balance, income, expense, top category, budget vs actual, tips	Financial overview.
Add Transaction	Type, category, amount, description, date	Record transaction.
Reports	Filters, charts, export	Analyze spending.
Budgets	Category limit, progress, alerts	Monitor budgets.
Insights	AI summary, patterns, tips, bookmark	Review insights.
Admin Panel	Users, categories, templates, statistics	Administration.

6. Database Design

Table	Important Fields	Purpose
users	user_id PK, name, email UNIQUE, password_hash, academic_year, monthly_savings_goal, created_at	Student account.
categories	category_id PK, name, type, is_default	Income/expense categories.
transactions	transaction_id PK, user_id FK, category_id FK, amount, type, description, ai_suggested_category, date, created_at	Transaction history.
budgets	budget_id PK, user_id FK, category_id FK, month, limit_amount	Monthly budgets.
insights	insight_id PK, user_id FK, month, summary_text, tip_text, generated_at	Monthly insight history.

7. Test Data

Sample local evaluation credentials. Replace these with the actual seeded credentials if the submitted project uses different accounts.

User Type	Email	Password	Role
Student	student@campuscoin.local	Student@123	Student
Administrator	admin@campuscoin.local	Admin@123	Admin

Sample Student Transactions

Date	Type	Category	Amount	Description
2026-09-01	Income	Allowance	30000	Monthly allowance
2026-09-02	Expense	Food	850	Campus cafe
2026-09-03	Expense	Transport	500	Ride / transport
2026-09-04	Expense	Academics	1200	Books and stationery
2026-09-05	Expense	Subscriptions	799	Streaming subscription
2026-09-06	Expense	Entertainment	1000	Outing
2026-09-07	Expense	Hostel/Rent	8000	Monthly hostel expense

8. Project Installation Instructions

This procedure assumes the documented React + Node.js/Express + MySQL implementation.

8.1 Prerequisites

- Windows 10/11 or compatible environment.
- Node.js LTS and npm.
- MySQL Server.
- VS Code or suitable IDE.
- Modern web browser.

8.2 Database Setup

1. Open MySQL/phpMyAdmin.
2. Create database named campus_coin.
3. Import the supplied SQL/schema file.
4. Confirm required tables exist.
5. Seed test accounts if required.

8.3 Backend Setup

1. Open backend directory.
2. Run npm install.
3. Create/configure environment variables.
4. Set database connection and authentication secret.
5. Configure optional AI API key if enabled.

6. Start the backend using the project's configured command.

8.4 Frontend Setup

1. Open frontend directory.
2. Run npm install.
3. Set API base URL if required.
4. Start frontend development server.
5. Open the displayed local URL.

8.5 Evaluation Sequence

1. Open home page.
2. Open sitemap.
3. Register or use student credentials.
4. Log income and expenses.
5. Check dashboard.
6. Create category.
7. Set monthly budget.
8. Open reports and filters.
9. Check saving tips and AI insights if enabled.
10. Log out.
11. Log in as administrator.
12. Check admin functions and statistics.
13. Log out.

9. Functional Test Cases

TC ID	Test	Expected Result
TC-01	Register valid student	Account created.
TC-02	Valid login	Dashboard opens.
TC-03	Invalid password	Access rejected.
TC-04	Add income	Income and balance update.
TC-05	Add expense	Expense and balance update.
TC-06	Edit transaction	Updated values appear.
TC-07	Delete transaction	Transaction removed and totals recalculate.
TC-08	Create category	Category becomes available.
TC-09	Set budget	Budget progress is shown.
TC-10	Generate report	Filtered report displays.
TC-11	AI category suggestion	Suggestion appears and can be overridden.
TC-12	Admin access	Admin functions available.
TC-13	Unauthorized data access	Other student's private data is blocked.
TC-14	Logout	Protected pages require login.

10. Assumptions and Scope Notes

- The SRS permits multiple technology stacks; React + Node.js/Express + MySQL is a documentation assumption.
- Exact API endpoint names are examples.
- AI categorization and monthly AI insights are optional.
- The application does not connect to real banking systems or process real payments.
- AI output is advisory and can be reviewed/overridden.
- Credentials listed are sample evaluation credentials.
- Documentation should not contain source code.

11. AI Usage Acknowledgement

AI-assisted tools may support design, UI/UX, debugging, learning and productivity. The SRS requires acknowledgement of AI tools used and states that the final solution should demonstrate the student's own design, development and understanding.

For this documentation workflow, ChatGPT was used to organize the documentation structure and draft explanatory material from the supplied SRS. The project team should review and verify the documentation against the actual implementation before submission.

12. Conclusion

Campus Coin is specified as a student-focused budgeting and expense-tracking web application combining authentication, transactions, categories, dashboards, reports, budgets, saving tips, optional AI assistance and administrative controls. The documentation translates the supplied SRS into an evaluator-oriented design, architecture, security, testing and installation guide while identifying assumptions separately.